

公共交通オープンデータチャレンジ2025 一次審査

プレゼン台本・想定質問集

3分間プレゼン台本

【オープニング：課題提起】 0:00-0:30（30秒）

[画面: ランディングページ]

「私たちは京都に住んでいます。

日々の生活で感じること——それは、オーバーツーリズムの影響でバスの所要時間が想定よりも大幅に長くなっていることです。

観光客の皆さんにとって、この長い移動時間は『待ち時間』でしかありません。

また、清水寺や金閣寺といった人気スポットへの集中も、混雑をさらに悪化させています。」

【解決アプローチの提示】 0:30-0:55（25秒）

「私たちはこの問題に対して、2つのアプローチで取り組みました。

ひとつは、**観光地の分散**。どのスポットが今空いているのかを可視化し、穴場への誘導を促します。

もうひとつは、**移動時間そのものを観光体験に変えること**。

AIが土地の歴史や文化を語りかけることで、長い移動時間も旅の一部になります。

これが『Path』です。」

【デモ開始】 0:55-1:00（5秒）

[操作: 「使い方を見る」をクリック]

「実際にお見せします。」

【デモ Step 1: 混雑度の可視化】 1:00-1:25（25秒）

[画面: スポット一覧]

「まず、周辺のスポットが混雑度とともに表示されます。

清水寺は混雑度5、伏見稲荷も5。一方、赤山禅院は混雑度1です。

行き先が決まっていない方は、これを見て空いているスポットを選べます。

これにより、**観光地の分散**が自然に促されます。」

【デモ Step 2: ルート検索】 1:25-1:45（20秒）

[画面: ルート選択]

「赤山禅院を選ぶと、GTFSデータをもとに複数のルートが提案されます。

京都駅から約53分、230円。バスと徒歩を組み合わせたルートです。

所要時間は長いですが、ここからがPathの真価です。」

【デモ Step 3: AIガイド ★メイン】 1:45-2:30（45秒）

[画面: ナビゲーション画面、シミュレーション開始]

「ナビゲーションを開始します。」

[操作: シミュレーションを進める]

「バスに乗車すると、AIガイドが始まります。」

[操作: AIガイド再生]

（音声を10～15秒流す）

「このように、移動中にその土地の歴史や見どころをAIが語りかけます。

AIはオープンデータの説明文を元に行っているため、嘘の情報を生成しません。

すでに行きたい場所が決まっている方にとっても、長くて退屈だった移動時間が観光体験に変わります。

さらに、ガイドを聞くことで**プラン外のスポットにも興味湧く**——そんな体験を提供します。」

【クロージング】 2:30-3:00（30秒）

[画面: そのまま]

「まとめます。

Pathは、**混雑度の可視化による観光地分散**と、**AIガイドによる移動時間の観光体験化**という2つのアプローチで、オーバーツーリズムの課題に取り組んでいます。

使用したオープンデータは、京都市バス・地下鉄のGTFS、KDDI人流データ、京都府観光施設一覧です。

すべてオープンデータのみで実現しています。

ご清聴ありがとうございました。」

時間配分サマリー

時間	セクション	秒数
0:00-0:30	オープニング（課題提起：京都在住者として実感する問題）	30秒
0:30-0:55	2つの解決アプローチ	25秒
0:55-1:00	デモ開始	5秒
1:00-1:25	混雑度の可視化（観光地分散）	25秒
1:25-1:45	ルート検索	20秒
1:45-2:30	AIガイド ★メイン（移動時間の観光体験化）	45秒
2:30-3:00	クロージング（まとめ＋データ紹介）	30秒

キーマッセージ（伝えるべき3つのポイント）

- 1. 京都在住者として課題を実感している（説得力）
- 2. 観光地分散 + 移動時間の価値化という2軸の解決策
- 3. すでにプランが決まっている人にも、決まっていない人にも価値がある

想定質問と回答集（5分間質疑応答用）

【カテゴリ1: オープンデータ関連】

Q1-1: 使用したオープンデータを教えてください

簡潔版（30秒）

「3つのオープンデータを使用しています。京都市バス・地下鉄のGTFSデータ、KDDI Location Analyzerの人流データ、京都府観光施設一覧です。これらを組み合わせて、混雑予測とルート検索を

実現しています。」

詳細版（1分）

「主に3つです。

1つ目は**京都市バス・地下鉄のGTFS**。停留所の位置、路線の経路形状、時刻表を抽出しています。元データは数GBありますが、ビルド時に必要な情報だけをJSON形式に前処理し、約2MBに圧縮しています。

2つ目は**KDDI Location Analyzerの人流データ**。京都市がオープンデータとして公開しているもので、2019～2021年の3年間の月別・時間帯別・1kmメッシュ別の滞在人口です。

3つ目は**京都府観光施設一覧**。約300スポットの名称、位置、説明文を使用しています。」

Q1-2: GTFSデータはどのように処理していますか？

回答

「GTFSは複数のCSVファイルで構成されていますが、そのままではWebアプリで扱うには重すぎます。

そこでビルド時にNode.jsスクリプトで前処理を行い、2つのJSONファイルに変換しています。

1つは `kyoto-stops-graph.json` ——停留所間の接続関係をグラフ構造で表現したもので、経路探索に使用します。

もう1つは `kyoto-bus-routes.json` ——路線の形状データで、地図上にルートを描画する際に使用します。

この前処理により、元データ数GBを約2MBまで圧縮し、クライアントサイドで高速に動作させています。」

Q1-3: 人流データの期間が2019～2021年ですが、最新データではないのでは？

回答

「ご指摘の通り、現時点で公開されている京都市のオープンデータが2019～2021年のものです。

ただし、このデータは**月別・時間帯別・曜日別**の統計的パターンを捉えるには十分有効です。たとえば「11月の紅葉シーズンの休日午後は混む」といった傾向は年によって大きく変わりません。

リアルタイムデータとの連携は今後の拡張として検討しています。たとえば、京都市バスのバスロケーションAPIや、人流のリアルタイムAPIが公開されれば、即座に組み込める設計になっています。」

【カテゴリ2: AI技術関連】

Q2-1: AIモデルは何を使っていますか？

回答

「OpenRouter経由で2つのモデルを使用しています。

メインは**GLM-4.5-Air**、フォールバックとして**Gemini 2.0 Flash**です。いずれも無料で利用できるモデルです。

OpenRouterを選んだ理由は、複数モデルへの切り替えが容易で、特定のAPIに依存しない設計にできるからです。将来的にモデルが変わっても、エンドポイントの変更だけで対応できます。」

Q2-2: AIのハルシネーション（嘘の情報生成）はどう防いでいますか？

簡潔版（30秒）

「RAGアーキテクチャを採用しています。AIには『新しい情報を生成する』のではなく、『オープンデータの説明文を、ガイド風の語り口に変換する』という役割だけを与えています。事実はずべてオープンデータから取得し、AIは表現変換のみを担当します。」

詳細版（1分30秒）

「ハルシネーション対策は設計の中核です。

一般的なAIアプリは『質問に対して回答を生成する』設計ですが、Pathは違います。

Pathでは、京都府観光施設一覧の**公式説明文をプロンプトに埋め込み**、AIには以下の指示だけを与えます：

1. 『～である』調を『～ですね』調のガイド口調に変換
2. 専門用語を観光客向けに平易化
3. 説明文に書かれていないことは話さない

つまり、AIの役割は**『事実の生成』ではなく『表現の変換』**に限定しています。

これにより、AIが嘘の歴史や存在しない建物を語ることを構造的に防いでいます。

実際のプロンプト例をお見せすると、こうなっています：

あなたは京都観光ガイドです。
以下の公式情報をもとに、親しみやすい口調で案内してください。
公式情報に書かれていないことは話さないでください。

【公式情報】

赤山禅院は、平安時代に創建された天台宗の寺院で...

このように、AIには『何を話すか』の自由度を与えず、『どう話すか』だけを任せています。」

Q2-3: 音声合成はどのように実装していますか？

回答

「ブラウザ標準の**Web Speech API**を使用しています。

具体的には `SpeechSynthesisUtterance` で日本語音声を生成し、ブラウザのネイティブ機能で読み上げます。

外部の音声合成APIを使わない理由は、コスト削減とオフライン対応です。Web Speech APIはブラウザに内蔵されているため、ネットワークが不安定な観光地でも音声再生が可能です。」

Q2-4: AIガイドの生成タイミングは？リアルタイムですか？

回答

「ナビゲーション開始時にバックグラウンドで生成を開始し、**移動フェーズごとにキャッシュ**しています。

具体的には：

- バス停へ歩くフェーズ
- バス乗車中フェーズ
- 目的地へ歩くフェーズ

この3つのガイド文を並列で生成し、localStorageに保存します。

一度生成したガイドは保存されるため、同じスポットを再訪問した場合はAPI呼び出しなしで即座に再生できます。これにより、API呼び出し回数を大幅に削減し、コストを抑えています。」

【カテゴリ3: 混雑度計算関連】

Q3-1: 混雑度はどのように計算していますか？

簡潔版（30秒）

「偏差値ベースの5段階評価です。人流データから各メッシュの滞在人口を取得し、観光スポットが存在するメッシュのみを母集団として偏差値を計算しています。偏差値42以下がレベル1、58以上がレベル5です。」

詳細版（1分30秒）

「混雑度の計算には**Tスコア（偏差値）**を使用しています。

手順は以下の通りです：

1. 人流データから、指定された月・時間帯・曜日の滞在人口を各メッシュごとに取得
2. **観光スポットが存在するメッシュのみ**を母集団として抽出
3. 母集団の平均と標準偏差を計算
4. 各スポットの位置のメッシュの人口を偏差値に変換
5. 偏差値を5段階に分類

なぜ『観光スポットが存在するメッシュのみ』を母集団にするかというと、京都は山間部と繁華街で人口密度の差が極端だからです。

全メッシュを母集団にすると、山間部の寺院はすべてレベル1になり、意味のある比較ができません。観光地同士での相対評価にすることで、『今日はどこが比較的空いているか』が分かるようになります。

具体的な閾値は：

- レベル1（快適）：偏差値42以下
- レベル2：42～48
- レベル3：48～52
- レベル4：52～58
- レベル5（混雑）：58以上

です。」

Q3-2: 1kmメッシュとは何ですか？

回答

「JIS X 0410で規定された**日本標準地域メッシュ**のことです。

日本全国を緯度・経度に基づいて格子状に分割したもので、1kmメッシュは約1km四方の区画を表します。各メッシュには8桁のコードが割り当てられています。

このコードを使うことで、スポットの緯度経度からどのメッシュに属するかを計算し、そのメッシュの人流データを取得できます。

Pathでは、この変換をクライアントサイドで高速に行うための関数を実装しています。」

Q3-3: リアルタイムの混雑度は取得できますか？

回答

「現時点ではリアルタイムデータは使用していません。過去の統計データによる『予測』です。

ただし、設計上はリアルタイムAPI対応が可能です。 `humanFlowService.ts` の `getCongestionLevel` 関数を差し替えるだけで、外部APIからのリアルタイムデータを組み込めます。

将来的に京都市がリアルタイム人流APIを公開したり、NTTドコモやKDDIのリアルタイムデータが利用可能になれば、即座に対応できる設計になっています。」

【カテゴリ4: ルート検索関連】

Q4-1: ルート検索のアルゴリズムを教えてください

回答

「2段階のアルゴリズムを採用しています。

第1段階：幾何学的フィルタリング

出発地と目的地の周囲3～5km以内のバス停を抽出し、有望な候補を絞り込みます。これにより、全停留所を探索する必要がなくなり、計算コストを大幅に削減しています。

第2段階：詳細計算

候補となるルートについて、以下を計算します：

- OSRM（Open Source Routing Machine）による徒歩経路と所要時間
- GTFSの時刻表との照合による乗車時間と待ち時間
- 乗り換えが必要な場合の接続確認

最終的に、所要時間・料金・乗り換え回数を比較できる複数のルート候補を返します。」

Q4-2: OSRMとは何ですか？

回答

「**Open Source Routing Machine**の略で、オープンソースの経路検索エンジンです。

OpenStreetMapのデータをもとに、徒歩・自転車・自動車の経路計算ができます。Pathでは徒歩経路の計算に使用しています。

公開されているパブリックAPIを利用していますが、将来的にはセルフホストして完全にオフライン対応することも可能です。」

Q4-3: なぜバス優先なのですか？地下鉄のほうが速いのでは？

回答

「おっしゃる通り、移動速度では地下鉄が優れています。

しかし、Pathのコンセプト『Travel as a Story』を考えると、**バスには地下鉄にない価値**があります。

バスは車窓から風景が見えます。AIガイドが『左手に見えるのは二条城です』と語りかけたとき、実際に目で確認できる。この視覚情報とガイドのリンクが、移動を観光体験に変える鍵です。

地下鉄は景色が見えないため、ガイドを聞いても視覚的な体験が伴いません。

もちろん、地下鉄ルートも選択肢として表示しますが、Pathとしてはバスを推奨する設計になっています。」

Q4-4: 終バス後の対応はどうなっていますか？

回答

「終バス後にルート検索を行った場合、翌朝の始発を提案する設計になっています。

GTFSの時刻表データから運行終了時刻を判定し、該当する便がない場合は翌日の最初の便を検索します。

ユーザーには『本日の運行は終了しています。翌朝〇時の便をご案内します』といったメッセージを表示します。」

【カテゴリ5: システムアーキテクチャ関連】

Q5-1: 技術スタックを教えてください

回答

「フロントエンドはReact 19とTypeScript、ビルドツールはVite、スタイリングはTailwind CSSです。

地図表示にはLeafletとReact-Leaflet、3D表示にはThree.jsとReact Three Fiberを使用しています。

バックエンドは持たず、Vercelのサーバーレス環境でホスティングしています。AI APIはOpenRouter経由で呼び出しています。

この構成により、サーバー運用コストをゼロに抑えつつ、高速なユーザー体験を実現しています。」

Q5-2: オフラインで動作しますか？

回答

「AIガイドの生成以外はオフラインで動作します。

人流データ、時刻表データ、スポット情報はすべてビルド時にJSONファイルに変換してアプリに同梱しています。地図タイルはキャッシュされていれば表示可能です。

一度生成したAIガイドもlocalStorageに保存されるため、再訪問時はオフラインで再生できます。

PWA（Progressive Web App）対応も行っており、スマートフォンのホーム画面に追加してネイティブアプリのように使用できます。」

Q5-3: なぜReact 19を選んだのですか？

回答

「React 19の新機能、特に**Concurrent Rendering**と**Suspense**の改善が、Pathのユーザー体験に直接寄与するからです。

ルート検索やAIガイド生成は時間がかかる処理ですが、Concurrent Renderingにより、処理中もUIがブロックされず、スムーズな操作感を維持できます。

また、チーム内でReactの経験があったことも選定理由です。」

Q5-4: 3D表示は何のために使っていますか？

回答

「現在地マーカーを3Dで表示しています。

従来の2Dマーカーだと、ユーザーがどちらを向いているか分かりにくいですが、3Dキャラクターを使うことで**進行方向が直感的に分かります**。

また、バス乗車中はキャラクターがバスの3Dモデルに切り替わり、移動フェーズが視覚的に明確になります。

技術的にはReact Three FiberとThree.jsで実装しており、ブラウザのWebGLで動作するため、追加のプラグインは不要です。」

【カテゴリ6: 実用化・拡張性関連】

Q6-1: 他の都市への展開は可能ですか？

回答

「GTFSデータが公開されている都市であれば展開可能です。

現在、日本では東京、大阪、名古屋など多くの都市でGTFSが公開されています。

課題は人流データです。京都市のようにオープンデータとして公開している自治体は限られています。ただし、統計的な混雑予測データがなくても、ルート検索とAIガイドの機能は動作します。

将来的には、オープンデータの整備状況に応じて段階的に展開地域を広げていく計画です。」

Q6-2: 実際のユーザーテストは行いましたか？

回答

「現時点では開発チーム内でのテストにとどまっています。

チュートリアル機能を使って、初見のユーザーでも操作できるかを検証しました。位置シミュレーター機能により、実際に移動しなくてもナビゲーション体験を試すことができます。

今後は京都市内での実地テストを予定しています。」

Q6-3: API利用料金はどうなっていますか？持続可能ですか？

回答

「現在使用しているOpenRouterのGLM-4.5-AirとGemini 2.0 Flashは**無料枠**で動作しています。

また、AIガイドはlocalStorageにキャッシュするため、同じスポットへの再訪問ではAPI呼び出しが発生しません。

OSRMも公開APIを使用していますが、利用量が増えた場合はセルフホストに切り替えることで、外部依存をゼロにできます。

サーバーレス環境（Vercel）も無料枠で運用しているため、現時点での運用コストはほぼゼロです。」

Q6-4: 多言語対応は考えていますか？

回答

「設計上は対応可能です。

AIガイドのプロンプトに『英語で案内してください』と指定するだけで、英語ガイドを生成できます。Web Speech APIも多言語対応しています。

UIの多言語化はi18nライブラリを導入することで実現できます。インバウンド観光客への展開を考えると、優先度の高い拡張です。」

【カテゴリ7: コンセプト・社会実装関連】

Q7-1: 既存のナビアプリとの違いは何ですか？

回答

「大きく2点あります。

1つ目は**混雑度の可視化**。Google MapsやYahoo!乗換案内は混雑度を考慮しません。Pathは人流データを使って、どのスポットが今空いているかを表示し、観光地の分散を促します。

2つ目は**AIガイドによる移動時間の価値化**。既存アプリは移動を『手段』として扱いますが、Pathは移動を『体験』に変えます。長い移動時間も、AIが土地の歴史を語ることで、観光の一部になります。」

Q7-2: オーバーツーリズム解決にどう貢献しますか？

回答

「2つのアプローチで貢献します。

空間的分散：混雑度を可視化することで、清水寺や金閣寺に集中している観光客を、赤山禅院のような穴場スポットへ誘導します。

時間的許容：AIガイドによって移動時間が苦痛でなくなれば、混雑するピーク時間帯を避けて、空いている時間帯に移動するインセンティブが生まれます。

どちらも、観光客の行動を強制するのではなく、**情報提供によって自発的な分散を促す**というアプローチです。」

Q7-3: ターゲットユーザーは誰ですか？

回答

「3つのユーザー層を想定しています。

1つ目は**初めて京都を訪れる観光客**。どこに行けばいいかわからない方に、混雑度を見ながらスポットを選んでもらえます。

2つ目は**リピーター**。有名スポットは行ったことがあるので、AIガイドで紹介される穴場スポットに興味を持ってもらえます。

3つ目は**移動に不安がある方**。高齢者や土地勘のない方でも、ナビゲーションとガイドがあれば安心して移動できます。」

【カテゴリ8: 公共交通オープンデータ関連】★重要

Q8-1: GTFSデータを使ってみて、課題や要望はありますか？

回答

「いくつかあります。

1つ目は**データサイズ**です。京都市バスのGTFSは数GBあり、Webアプリでそのまま使うには重すぎます。私たちはビルド時に前処理して約2MBに圧縮しましたが、公式に軽量版が提供されると開発者にとって助かります。

2つ目は**リアルタイムデータ（GTFS-RT）の公開**です。現在の静的GTFSでは遅延情報が反映できません。バスロケーションAPIやGTFS-RTが公開されれば、より正確なナビゲーションが可能になります。

3つ目は**運賃データの充実**です。GTFSのfare_attributes.txtが提供されていると、料金計算の精度が上がります。」

Q8-2: GTFS-RT（リアルタイムデータ）への対応は考えていますか？

回答

「設計上は対応可能です。

現在のルート検索ロジックは、時刻表ベースの『予定時刻』を使っていますが、GTFS-RTのTripUpdateやVehiclePositionを取得すれば、リアルタイムの遅延情報を反映できます。

具体的には、routeService.ts の時刻表照合部分を拡張し、リアルタイムフィードから遅延分を加算する処理を追加します。

京都市バスがGTFS-RTを公開した際には、すぐに対応したいと考えています。」

Q8-3: 公共交通事業者との連携は考えていますか？

回答

「ぜひ連携したいと考えています。

具体的には：

1. **データ連携**：バスロケーションAPIや混雑度データの提供を受けられれば、リアルタイム性が大幅に向上します
2. **コンテンツ連携**：沿線の観光情報や車内アナウンスとの連動があれば、ガイド体験がより豊かになります

3. **実証実験**：観光客の行動変容（混雑回避効果）を測定するための実証実験を共同で行いたいです
事業者様にとっても、**観光客の分散による混雑緩和と公共交通の利用促進**というメリットがあると考えています。」

Q8-4: このアプリで公共交通の利用は促進されますか？

回答

「はい、2つの観点で促進されと考えています。

1つ目は**心理的ハードルの低減**です。土地勘のない観光客にとって、バスは路線が複雑で乗りにくい交通手段です。Pathはルート検索とナビゲーションで、その不安を解消します。

2つ目は**移動体験の価値向上**です。従来、バス移動は『目的地に着くまでの我慢の時間』でした。AIガイドによって移動自体が楽しくなれば、『バスに乗りたい』というポジティブな動機が生まれます。

結果として、タクシーやレンタカーではなく公共交通を選ぶ観光客が増えると期待しています。」

Q8-5: GTFSの標準仕様にはどの程度準拠していますか？

回答

「京都市が公開しているGTFSデータは、Google Transit Feed Specificationに準拠しています。

私たちのアプリでは、以下のファイルを使用しています：

- stops.txt ：停留所の位置情報
- routes.txt ：路線情報
- trips.txt ：便情報
- stop_times.txt ：時刻表
- shapes.txt ：路線の形状（地図描画用）
- calendar.txt / calendar_dates.txt ：運行日

標準仕様に準拠しているため、他の都市のGTFSデータでも同じロジックで動作します。これがGTFS標準化の大きなメリットです。」

Q8-6: MaaS（Mobility as a Service）との連携は考えていますか？

回答

「将来的には連携を視野に入れています。

Pathは現在、情報提供に特化していますが、MaaSプラットフォームと連携すれば：

- ・ **決済連携**：アプリ内でバス乗車券を購入
- ・ **マルチモーダル**：バス・地下鉄・シェアサイクル・タクシーを組み合わせたルート提案
- ・ **サブスクリプション**：観光客向けフリーパスとの連動

といった拡張が可能です。

まずは情報提供アプリとしての完成度を高め、その後MaaS連携を検討したいと考えています。」

Q8-7: バス以外の交通モード（鉄道、タクシーなど）への対応は？

回答

「現在は京都市営地下鉄にも対応しています。GTFSデータが公開されているためです。

JRや私鉄については、GTFSデータの公開状況によります。公開されていれば、同じロジックで統合可能です。

タクシーについては、GTFSの枠組みでは扱いにくいですが、GBFS（シェアモビリティ向け標準）やタクシー配車APIとの連携は技術的に可能です。

Pathのコンセプト『Travel as a Story』を考えると、**景色が見える交通手段**（バス、路面電車、観光船など）が最も相性が良いと考えています。」

Q8-8: GTFSデータの更新にはどう対応していますか？

回答

「現時点では手動でデータを更新しています。

京都市がGTFSを更新した際に、前処理スクリプトを再実行してJSONファイルを再生成し、アプリを再デプロイします。

将来的には、GTFSの更新を自動検知し、CI/CDパイプラインで自動的に前処理・デプロイする仕組みを構築したいと考えています。

GTFSデータのバージョン管理や更新通知の仕組みがあると、開発者としては非常に助かります。」

Q8-9: オープンデータの品質についてどう感じましたか？

回答

「京都市のGTFSデータは品質が高いと感じました。

停留所の位置精度、時刻表の正確性、路線形状の詳細さ、いずれも十分実用レベルです。

一方で、いくつか改善の余地を感じた点もあります：

1. **運賃データ**：fare_attributes.txtがより詳細だと、複雑な運賃体系（均一運賃区間、乗り継ぎ割引など）を正確に計算できます
2. **多言語対応**：stops.txtやroutes.txtに英語名があると、インバウンド対応が容易になります
3. **アクセシビリティ情報**：wheelchair_boardingなどの情報が充実すると、バリアフリー対応が可能になります」

Q8-10: 公共交通オープンデータの普及に向けて、何が必要だと思いますか？

回答

「3つあると考えています。

1つ目は**成功事例の可視化**です。オープンデータを使ったアプリが実際に利用され、公共交通の利用促進につながった事例を示すことで、事業者のデータ公開へのモチベーションが高まります。

2つ目は**開発者コミュニティの育成**です。GTFSを扱えるエンジニアが増えれば、より多くのアプリが生まれます。ハンズオンやサンプルコードの充実が重要です。

3つ目は**標準化の推進**です。GTFSに加え、GTFS-RT、GBFS、MaaS Japan仕様など、標準フォーマットが普及すれば、データ連携のコストが下がります。

このチャレンジのような取り組みが、まさにその推進力になっていると感じています。」

Q8-11: 他のオープンデータチャレンジ応募作品との差別化ポイントは？

回答

「Pathの差別化ポイントは**『移動体験の価値化』**です。

多くのナビアプリは『A地点からB地点への最短・最安ルート』を競います。これは重要ですが、コモディティ化しています。

Pathは、**移動時間そのものを観光体験に変える**という新しい価値を提案しています。

公共交通のオープンデータを『効率化』だけでなく『体験向上』に活用する——これがPathの独自性です。

また、**複数のオープンデータを組み合わせている**点も特徴です。GTFS単体ではなく、人流データ、観光施設データを掛け合わせることで、混雑回避という社会課題に取り組んでいます。」

Q8-12: 京都以外の観光地でも同じアプローチは有効ですか？

回答

「有効だと考えています。

オーバーツーリズムは京都だけの問題ではありません。鎌倉、奈良、金沢、広島など、多くの観光地で同様の課題があります。

Pathのアプローチ——**混雑度可視化による分散と移動時間の観光体験化**——は、観光地であれば共通して適用できます。

必要なのは：

- GTFSデータ（公共交通）
- 人流データ（混雑度）
- 観光施設データ（スポット情報）

この3つがオープンデータとして公開されていれば、同じ仕組みを横展開できます。

Pathを京都での成功事例として確立し、他の観光地へ展開していくことが目標です。」

Q8-13: 公共交通事業者にとってのメリットは何ですか？

回答

「3つのメリットがあります。

1. 混雑緩和

観光客が特定の路線・時間帯に集中すると、地元住民の利用に支障が出ます。Pathで観光客を分散させることで、混雑が緩和されます。

2. 利用促進

土地勘のない観光客は、バスを避けてタクシーを使いがちです。Pathがバス利用のハードルを下げれば、乗客数の増加につながります。

3. 観光資源としてのブランディング

『バスに乗ること自体が観光体験になる』というコンセプトは、公共交通の新しい価値を提案しています。観光地の交通事業者にとって、差別化ポイントになり得ます。」

Q8-14: データの二次利用や派生データの公開は考えていますか？

回答

「はい、検討しています。

具体的には：

1. 前処理済みGTFSデータの公開

私たちが作成した `kyoto-stops-graph.json` などの軽量化データを、他の開発者が使えるよう公開することを検討しています。

2. 混雑度APIの提供

観光スポットの混雑度を返すAPIを公開すれば、他のアプリでも活用できます。

3. 利用ログの匿名化公開

ユーザーの検索傾向（人気スポット、時間帯別アクセスなど）を匿名化して公開すれば、観光施策の参考になります。

オープンデータで受けた恩恵を、オープンデータとして還元していきたいと考えています。」

【カテゴリ9: 審査員が聞きそうな鋭い質問】

Q9-1: このアプリは本当にオーバーツーリズムを解決できますか？

回答

「正直に申し上げますと、**Path**だけでオーバーツーリズムを解決することはできません。

オーバーツーリズムは複合的な社会課題であり、宿泊施設の分散、入場制限、価格政策など、多面的なアプローチが必要です。

Pathが担えるのは、**情報提供による行動変容の促進**です。

『今どこが空いているか』『どのルートが快適か』という情報を提供することで、観光客が自発的に分散行動を取るきっかけを作ります。

小さな貢献ですが、テクノロジーでできることとして取り組んでいます。」

Q9-2: AIガイドは本当に必要ですか？テキスト表示だけでは？

回答

「テキスト表示だけでも情報は伝わりますが、**音声ガイドには3つの利点**があります。

1つ目は**ハンズフリー**。バスの中でスマホ画面を見続けるのは難しいですが、音声なら車窓を眺めながら聞けます。

2つ目は**没入感**。プロのガイドに案内されているような体験は、テキストでは得られません。

3つ目は**アクセシビリティ**。視覚に障害がある方、小さい文字が読みにくい高齢者の方にも情報を届けられます。

移動時間を『体験』に変えるためには、音声ガイドが不可欠だと考えています。」

Q9-3: 位置情報の精度は十分ですか？バスの中でGPSは正確に動きますか？

回答

「ご指摘の通り、GPS精度は課題の一つです。

バスの中では、建物による遮蔽やマルチパスの影響で、位置精度が低下することがあります。

現在の対処として：

1. **スムージング処理**：急激な位置変化を抑制し、軌跡を滑らかにしています
2. **ルートスナッピング**：検索したルート上に位置を補正しています
3. **移動フェーズの手動切り替え**：GPSに頼らず、ユーザーが『乗車した』『降車した』を手動で切り替えることも可能です

将来的には、GTFS-RTのVehiclePositionデータと照合することで、バス位置を正確に把握できるようになります。」

Q9-4: ユーザーはわざわざこのアプリを使いますか？ Google Mapsで十分では？

回答

「Google Mapsは素晴らしいアプリですが、**観光体験を提供するものではありません。**

Google Mapsは『移動の効率化』に特化しています。最短ルート、最安ルート、乗り換え案内——どれも正確です。

しかし、Pathが提供するの****『移動の楽しさ』****です。

たとえば：

- 『今、バスで通過しているこの通りは、平安時代には〇〇だった』
- 『左手に見える山は比叡山。最澄が開いた延暦寺があります』

こうした体験は、Google Mapsでは得られません。

『効率』ではなく『体験』を求める観光客——特に京都のような歴史ある街を訪れる方——にとって、Pathは価値があると考えています。」

Q9-5: チーム構成と開発期間を教えてください

回答

「チームは○名で構成されています。[※実際のチーム構成に合わせて回答]

開発期間は約○ヶ月です。[※実際の期間に合わせて回答]

役割分担としては：

- 企画・UI/UXデザイン
- フロントエンド開発（React、地図、3D）
- データ処理（GTFS前処理、人流データ分析）
- AIプロンプト設計

全員が京都在住または京都に縁があり、オーバーツーリズムの問題を日常的に感じていたことが、このアプリを作るきっかけになりました。」

Q9-6: 今後の開発ロードマップを教えてください

回答

「3つのフェーズを考えています。

フェーズ1（短期）：

- ユーザーテストの実施と改善
- 対応スポット数の拡充
- 多言語対応（英語）

フェーズ2（中期）：

- GTFS-RT対応（リアルタイム遅延情報）
- リアルタイム人流データとの連携
- 他の観光都市への横展開

フェーズ3（長期）：

- 公共交通事業者との連携（データ、決済）
- MaaSプラットフォームとの統合
- AR機能の追加（カメラを通じた観光案内）

まずはフェーズ1を着実に進め、実際の観光客に使ってもらえるアプリに仕上げていきます。」

質問対応のコツ

状況	対応
技術的に深い質問	「詳細版」の回答を使う。実装の具体例（ファイル名、関数名）を出すと説得力が増す
時間が限られている	「簡潔版」の回答を使う。30秒以内に収める
分からない質問	「現時点では検討中です」「今後の拡張として考えています」と正直に答える
批判的な質問	課題を認めた上で、対策や今後の計画を説明する

事前準備チェックリスト

- ☐ アプリをローカルで起動（npm run dev）
- ☐ OpenRouter APIキーの動作確認
- ☐ 音声出力テスト（スピーカー/ヘッドセット）
- ☐ 画面共有テスト（解像度・フレームレート）
- ☐ シミュレーション速度の調整（速すぎないか）
- ☐ 台本を手元に用意
- ☐ 時間を測ってリハーサル（3分に収まるか）
- ☐ デモで使うルート（京都駅→赤山禅院）の動作確認